



Documentación Evolutiva del Proyecto: Body Paint

Grupo 6

1. Integrantes

<i>Nombre y Apellido</i>	<i>Mail</i>	<i>Legajo</i>	<i>Rol Interno (Opcional)</i>
Lautaro Oliviero	lutarooliviero44@gmail.com	17428	Scrum Master / Developer
Mauricio Encina	maurixmre@gmail.com	17360	Developer
Luciana Sassia	sassialuciana@gmail.com	17475	Developer

2. Enlaces del Proyecto

Repositorio en GitHub:

<https://github.com/LautiOliviero/TRABAJO-PR-CTICO-INTEGRADOR-TPI-MSII---GRUPO-6.git>

Proyecto en Jira:

<https://grupo6msll.atlassian.net/jira/software/projects/SCRUM/boards/1?atlOrigin=eyJpIjoiMjI0ODRIZGExZjVjNGUxODhIMjcwMTU5N2YwMjI2MDgiLCJwIjoiaW9>

3. Roles del Sistema y Responsabilidades

- **Cliente:** Usuario que navega el catálogo, arma carros de compras y confirma pedidos con pago contra entrega.
- **Vendedor:** Usuario interno de Body Paint que visualiza y gestiona los estados de los pedidos de los clientes.
- **Administrador de Productos:** Usuario interno encargado de mantener actualizado el catálogo de productos, gestionar el stock, cargar nuevos productos y visualizar las alertas de inventario.

(Nota: El rol del Product Owner (PO) es ejercido por los docentes de la cátedra).

4. Registro de Sprints

SPRINT 0

I. Objetivo del Sprint:

Realizar el análisis funcional y técnico del dominio 'Body Paint', configurar las herramientas de gestión (Jira y GitHub) y definir los criterios de aceptación y estimaciones para las User Stories del cliente y el MVP.

II. User Stories seleccionadas para el Sprint Backlog:

US-01: Cupón de Descuento

US-02: Gestión de Reclamos

US-03: Solicitud de Reenvío

III. Detalle de cada User Story:

US-01: Cupón de Descuento

- **Descripción:** Como cliente, quiero ingresar un cupón de descuento para obtener beneficios en mi compra.
- **Criterios de Aceptación:**

1. El sistema debe permitir el ingreso de un código de cupón en el carro de compras.

2. El descuento solo se aplica si el monto del pedido es mayor al descuento obtenido.

3. Al confirmar el pago, el cupón debe marcarse como usado y quedar asociado al pedido.

- **Estimación:** Requiere validación de reglas de negocio y actualización de montos en tiempo real.

- o Esfuerzo: 5 (Escala Fibonacci).

- o Complejidad: Media.

- o Incertidumbre: Baja.

US-02: Gestión de Reclamos

- **Descripción:** Como cliente, quiero realizar reclamos dentro de los 5 días posteriores a la entrega para solucionar problemas.

- **Criterios de Aceptación:**

- 1. El sistema debe habilitar la opción de reclamo solo hasta 5 días hábiles después de la entrega.

- 2. Si no hay reclamo en ese plazo, el pedido se marca como finalizado automáticamente.

- **Estimación:** Esfuerzo bajo ya que se reutilizará la lógica de validación de fechas de proyectos anteriores, aunque con incertidumbre media por la integración de feriados.

- o Esfuerzo: 3.

- o Complejidad: Baja.

- o Incertidumbre: Media (por el cálculo de días hábiles).

US-03: Solicitud de Reenvío

- **Descripción:** Como cliente, quiero solicitar el reenvío de productos para recibir lo faltante o defectuoso.

- **Criterios de Aceptación:**

- 1. El cliente puede elegir si el reenvío es por la totalidad o una parte del pedido.

2. La solicitud debe quedar sujeta a un análisis previo del vendedor.

- **Estimación:** Se considera de esfuerzo alto no solo por la lógica de selección de productos, sino por la incertidumbre crítica respecto a la disponibilidad de stock. Si el producto a reenviar no está disponible, el sistema debe contemplar un flujo alternativo (como nota de crédito o espera de reposición), lo que aumenta el riesgo técnico y de negocio.

- o Esfuerzo: 8.

- o Complejidad: Alta (implica lógica de logística y nuevos estados de pedido).

- o Incertidumbre: Alta.

IV. Resultado del Sprint:

Funcionalidades implementadas:

- Configuración del entorno de gestión: Setup de Jira Software con tablero Scrum y creación del repositorio organizacional en GitHub.
- Análisis funcional y técnico: Definición de Criterios de Aceptación y estimaciones Fibonacci para las historias de usuario asignadas.
- Definición de Stack Tecnológico: Selección de HTML/JS/Bootstrap y MockAPI como arquitectura base.

Funcionalidades pendientes o replanificadas:

El desarrollo del código fuente de las User Stories (US-01, US-02, US-03) se traslada al Sprint 3, priorizando en esta etapa la base estructural y técnica del MVP.

V. Decisiones relevantes tomadas por el equipo:

Decisiones técnicas:

1. Actualización de Stock en tiempo real (Persistencia): Se optó por una arquitectura basada en servicios externos mediante MockAPI, que permite la actualización automática de las cantidades de productos cada vez que se confirme un pedido. Esto garantiza que el Administrador visualice datos reales y actualizados, cumpliendo con el requisito de visibilidad del stock de forma inmediata.

2. Tecnología: Se utilizará un stack basado en HTML, CSS (Bootstrap) y JavaScript. Esta decisión se fundamenta en la experiencia previa del equipo con estas herramientas, lo que garantiza agilidad en el desarrollo del Frontend y asegura la entrega del MVP en los plazos establecidos, minimizando el riesgo técnico.

SPRINT 1

I. Objetivo del Sprint:

Configurar el entorno de desarrollo local, establecer la arquitectura de carpetas del proyecto y diseñar la integración inicial con la API externa (MockAPI) para renderizar el catálogo de productos de forma dinámica sobre una interfaz responsiva base.

II. Épicas y Tareas seleccionadas para el Sprint Backlog:

Épica: Desarrollo de la Base Estructural - Parte I: Infraestructura y Datos

Tarea 1: Configuración de estructura de directorios (/css, /js, /assets).

Tarea 2: Creación de `index.html` base con componentes comunes.

Tarea 3: Setup de recursos y datos iniciales en MockAPI.

Tarea 4: Desarrollo de la lógica de Fetch para carga de catálogo.

Tarea 5: Maquetado de Cards de productos con Bootstrap.

III. Detalle de la Épica y sus Tareas Internas:

Épica: Desarrollo de la Base Estructural - Parte I: Infraestructura y Datos

- **Descripción:** Configuración inicial del entorno de desarrollo, arquitectura de carpetas y establecimiento de la comunicación con la API externa. El objetivo es tener el catálogo de productos funcional y el esqueleto visual del sitio listo para recibir la lógica de negocio.

- **Criterios de Aceptación:**

1. Estructura Base: El proyecto debe contar con la arquitectura de directorios definida y una plantilla base (Boilerplate) con Navbar y Footer funcionales.
2. Conexión a MockAPI: Se deben haber creado las colecciones de clientes, productos y pedidos en la plataforma.
3. Catálogo Dinámico: Los productos mostrados en la galería deben consumirse mediante Fetch desde MockAPI (no deben estar harcodeados en el HTML).
4. Maquetado UI: La interfaz debe ser responsiva y utilizar componentes de Bootstrap para la visualización de productos y el contenedor del carrito.

- **Estimación:** Representa el esfuerzo inicial de setup, diseño de la base de datos simulada y la integración de la primera capa de datos con la interfaz.

o Esfuerzo Total Épica: 8 (Escala Fibonacci).

Tarea 1: Configuración de estructura de directorios

- **Descripción:** Creación y ordenamiento de las carpetas locales del proyecto (/css, /js, /assets) para asegurar una arquitectura limpia.
- **Estimación:** Esfuerzo mínimo de organización interna. Esfuerzo: 1 | Complejidad: Baja | Incertidumbre: Baja

Tarea 2: Creación de index.html base con componentes comunes

- **Descripción:** Armado del esqueleto HTML e integración de las librerías de Bootstrap con Navbar y Footer funcionales.
- **Estimación:** Esfuerzo bajo enfocado en la estructura visual base.

Esfuerzo: 1 | Complejidad: Baja | Incertidumbre: Baja

Tarea 3: Setup de recursos y datos iniciales en MockAPI

- **Descripción:** Configuración técnica de la plataforma externa y creación de las colecciones base (**users**, **products**, **orders**) con datos de prueba.
- **Estimación:** Requiere mapear correctamente los campos necesarios para las historias futuras. Esfuerzo: 2 | Complejidad: Baja | Incertidumbre: Baja

Tarea 4: Desarrollo de la lógica de Fetch para carga de catálogo

- **Descripción:** Implementación del script de JavaScript encargado de realizar la petición asíncrona (Fetch API) para obtener los productos de manera dinámica.
- **Estimación:** Implica manejo de asincronismo y control de errores en las peticiones. Esfuerzo: 2 | Complejidad: Media | Incertidumbre: Baja

Tarea 5: Maquetado de Cards de productos con Bootstrap

- **Descripción:** Diseño de la interfaz responsiva de la galería que recibe los datos de la API y los renderiza dinámicamente en pantalla.
- **Estimación:** Enfoque en estilos estructurados y visualización limpia de los datos inyectados. Esfuerzo: 2 | Complejidad: Baja | Incertidumbre: Baja

IV. Resultado del Sprint:

Funcionalidades implementadas:

- Estructura e Infraestructura Base: Inicialización exitosa de las tareas técnicas SCRUM-42, SCRUM-43, SCRUM-44, SCRUM-45 y SCRUM-46, completando el alcance definido para la primera parte de la base estructural.

Funcionalidades pendientes o replanificadas:

El desarrollo de la Base Estructural - Parte II (Lógica de Carrito y Checkout básico) permanece en el Backlog del proyecto para ser activado en el próximo periodo, manteniendo postergado el código de las historias de usuario principales (US-01, US-02, US-03) hasta contar con dichos cimientos.

V. Decisiones relevantes tomadas por el equipo:

Decisiones técnicas:

1. Postergación de lógica de almacenamiento local: Se decidió de manera unánime postergar la implementación de LocalStorage para sprints posteriores. Esto reduce la sobrecarga técnica actual del equipo y permite asegurar primero la estabilidad de las peticiones HTTP (Fetch) hacia la API.

2. Estrategia de Desarrollo Paralelo (Reparto de Tareas): Para mitigar el impacto de la restricción horaria del equipo (estudio y trabajo), la Base Estructural se atomizó en las 5 tareas internas asignadas en paralelo, garantizando que el setup de MockAPI y el maquetado del frontend avanzaran de forma independiente y coordinada.